

A 问题一：稳健黎曼伪迹筛选

```

1 # 第一问核心算法：黎曼几何质量指数计算与试次筛查
2 # 以下函数摘自第八步正式源程序，省略公共导入和参数定义，保留算法逻辑
3
4 # 对称正定矩阵进行谱分解，用于计算矩阵对数、平方根等矩阵函数
5 def matrix_function(matrix, function, positive=False):
6     values, vectors = eigh((matrix + matrix.T) / 2)
7     if positive:
8         values = np.maximum(values, 1e-12)
9     return (vectors * function(values)) @ vectors.T
10
11
12 # 计算单个试次的通道协方差矩阵，并加入微小正则项
13 def covariance_matrix(signals):
14     centered = signals - signals.mean(axis=1, keepdims=True)
15     covariance = centered @ centered.T / (centered.shape[1] - 1)
16     return covariance + 1e-6 * np.eye(centered.shape[0])
17
18
19 # 计算两个协方差矩阵之间的黎曼距离
20 def riemann_distance(first, second):
21     inverse_root = matrix_function(first, lambda x: 1 / np.sqrt(x), positive=True)
22     relative = inverse_root @ second @ inverse_root
23     eigenvalues = eigh((relative + relative.T) / 2, eigvals_only=True)
24     return np.linalg.norm(np.log(np.maximum(eigenvalues, 1e-12)))
25
26
27 # 迭代计算协方差矩阵组的黎曼均值
28 def riemann_mean(covariances):
29     log_mean = np.mean(
30         [matrix_function(cov, np.log, positive=True) for cov in covariances], axis=0
31     )
32     center = matrix_function(log_mean, np.exp)
33     for _ in range(30):
34         root = matrix_function(center, np.sqrt, positive=True)
35         inverse_root = matrix_function(center, lambda x: 1 / np.sqrt(x), positive=True)
36         delta = np.mean(
37             [matrix_function(inverse_root @ cov @ inverse_root, np.log, positive=True)
38              for cov in covariances],
39             axis=0,
40         )
41         if np.linalg.norm(delta, "fro") < 1e-8:
42             break
43         center = root @ matrix_function(delta, np.exp) @ root
44         center = (center + center.T) / 2
45     return center
46
47
48 # 依据黎曼距离和稳健标准分，迭代识别协方差矩阵内点
49 def robust_riemann_center(covariances):
50     keep = np.ones(len(covariances), dtype=bool)
51     for _ in range(4):
52         center = riemann_mean(covariances[keep])
53         distances = np.array([riemann_distance(cov, center) for cov in covariances])
54         reference = distances[keep]
55         median = np.median(reference)
56         mad = np.median(np.abs(reference - median))

```

```

57     z_score = (distances - median) / (1.4826 * mad + 1e-12)
58     updated = z_score <= 3
59     if np.array_equal(updated, keep):
60         break
61     keep = updated
62     return riemann_mean(covariances[keep]), keep
63
64
65 # 对指定频带进行四阶零相位带通滤波
66 def bandpass(data, sample_rate, low_hz, high_hz):
67     sos = butter(4, [low_hz, high_hz], btype="bandpass", fs=sample_rate, output="sos")
68     return sosfiltfilt(sos, data, axis=-1)
69
70
71 # 以稳健内点为参照，将异常特征转换为非负稳健标准分
72 def positive_robust_z(feature, inliers):
73     reference = feature[inliers]
74     median = np.median(reference)
75     mad = np.median(np.abs(reference - median))
76     z_score = (feature - median) / (1.4826 * mad + 1e-12)
77     return np.maximum(0, np.clip(z_score, -3, 6))
78
79
80 # 在质量指数排序曲线的前 40% 范围内，以最大相邻跳变确定拐点
81 def sqi_knee_threshold(sqi):
82     ordered = np.sort(sqi)
83     if len(ordered) < 2:
84         raise ValueError("At least two non-clipped trials are required to estimate SQI threshold")
85     search_end = min(len(ordered) - 1, max(3, int(len(ordered) * 0.4)))
86     return ordered[np.argmax(np.diff(ordered[:search_end + 1])) + 1]
87
88
89 # 按组别配置当前项目实际使用的频带、通道和异常度量
90 def feature_specs(is_group_a):
91     if is_group_a:
92         return [
93             ("RP1_F3F4_1_7_Frobenius", "low", (1, 2), "frobenius"),
94             ("RP2_Fz_1_7_Variance", "low", (0,), "variance"),
95             ("RP3_All_16_24_Trace", "high", (0, 1, 2), "trace"),
96             ("RP4_All_1_24_Riemann", "full", (0, 1, 2), "riemann"),
97         ]
98     return [
99         ("RP1_All_1_7_Riemann", "low", (0, 1, 2), "riemann"),
100         ("RP2_F3F4_16_24_Trace", "high", (1, 2), "trace"),
101         ("RP3_All_1_24_Riemann", "full", (0, 1, 2), "riemann"),
102     ]
103
104
105 # 计算各试次的频带异常特征、质量指数和组别阈值
106 def calculate_sqi(data, sample_rate, time, valid_indices, is_group_a):
107     epoch = (time >= EPOCH_START) & (time <= EPOCH_END)
108     if not epoch.any():
109         raise ValueError(f"No samples in the analysis window {EPOCH_START} - {EPOCH_END} s")
110
111     bands = {
112         "low": bandpass(data, sample_rate, 1, 7),
113         "high": bandpass(data, sample_rate, 16, 24),
114         "full": data,
115     }

```

```

116     feature_names, feature_values, normalized = [], [], []
117
118     for feature_name, band, channels, metric in feature_specs(is_group_a):
119         channel_indices = np.asarray(channels)
120         covariances = np.stack([
121             covariance_matrix(bands[band][trial, channel_indices][:, epoch])
122             for trial in valid_indices
123         ])
124         center, inliers = robust_riemann_center(covariances)
125
126         if metric == "frobenius":
127             values = np.linalg.norm(covariances, axis=(1, 2))
128         elif metric == "variance":
129             values = covariances[:, 0, 0]
130         elif metric == "trace":
131             values = np.trace(covariances, axis1=1, axis2=2)
132         else:
133             values = np.array([riemann_distance(cov, center) for cov in covariances])
134
135         feature_names.append(feature_name)
136         feature_values.append(values)
137         normalized.append(positive_robust_z(values, inliers))
138
139     features = np.column_stack(feature_values)
140     anomaly_score = np.mean(normalized, axis=0)
141     sqi_valid = np.exp(-anomaly_score)
142     knee = sqi_knee_threshold(sqi_valid)
143     threshold = knee if is_group_a else max(0.05, 0.6 * knee)
144     return feature_names, features, sqi_valid, threshold
145
146
147 # 削顶试次不参与质量指数计算；两类异常标记取并集作为最终剔除结果
148 valid_indices = np.flatnonzero(~drop_values.astype(bool))
149 feature_names, features, sqi_valid, threshold = calculate_sqi(
150     eeg, sample_rate, time, valid_indices, dataset_name.startswith("VisualCogA")
151 )
152 riemann_drop = np.zeros(len(trials), dtype=bool)
153 riemann_drop[valid_indices] = sqi_valid < threshold
154 final_drop = drop_values.astype(bool) | riemann_drop

```

B 问题二：视觉刺激正向脑电模型

```

1 # 问题二核心算法：视觉前端提供 B、shape、dir_left、dir_right 特征。
2 # 以下保留皮层群体动力学、功能源映射和头皮电极正向投影。
3
4 W = {"w_ee": 1.4, "w_ei": 1.1, "w_ie": 1.0, "w_ii": 0.8,
5      "g_p": 2.2, "g_q": 1.2, "tau_e": 18.0, "tau_i": 10.0,
6      "tau_i_ratio": 0.55, "delay_early": 8.0, "delay_shape": 33.0,
7      "feed_e": 0.9, "feed_i": 0.45}
8 SIGMA, RADIUS = 0.33, 0.09
9 ELECTRODES = np.array([[ -35, 55, 62], [0, 66, 61], [35, 55, 62]], float)
10 REFERENCES = np.array([[ -70, -50, 5], [70, -50, 5]], float)
11 SOURCES = np.array([[ -10, -58, 45], [10, -58, 45], [-36, -25, 50],
12                    [36, -25, 50], [0, -34, 56]], float)
13
14
15 def activation(x, beta, threshold):
16     z = np.clip(beta * (np.asarray(x) - threshold), -60, 60)
17     f0 = 1 / (1 + np.exp(beta * threshold))
18     return np.clip((1 / (1 + np.exp(-z)) - f0) / (1 - f0), 0, 1)
19
20
21 def project_features(frontend, scales=None):
22     # 空间图按左右视野平均；两个模板偏好通道保持分离。
23     def halves(x):
24         mid = x.shape[1] // 2
25         return np.stack([x[:, :mid].mean((0, 1)), x[:, mid:].mean((0, 1))]) / np.sqrt(2)
26     drives = np.stack([halves(frontend["B"]), halves(frontend["shape"]),
27                       [frontend["dir_left"], frontend["dir_right"]]]).astype(np.float32)
28     return drives if scales is None else drives / np.asarray(scales)[: , :, None]
29
30
31 def delay_signal(x, time_ms, delay_ms):
32     target = time_ms - delay_ms
33     rows = np.asarray(x).reshape(-1, len(time_ms))
34     return np.stack([np.interp(target, time_ms, row, left=0, right=0)
35                     for row in rows]).reshape(x.shape)
36
37
38 def history_at(x, step, delay_samples):
39     # 只从已计算状态取样，并对分数延迟作线性插值。
40     whole = int(np.floor(delay_samples))
41     frac, index = delay_samples - whole, step - whole
42     if index < 0 or (index == 0 and frac > 1e-12):
43         return np.zeros(x.shape[:-1], dtype=x.dtype)
44     current = x[... , index]
45     previous = x[... , index - 1] if index else np.zeros_like(current)
46     return (1 - frac) * current + frac * previous
47
48
49 def wilson_cowan(drives, time_ms, tau_s=40.0, g_i=1.0):
50     # 三群体分别表示早期对比、空间构型和三角模板偏好。
51     dt = np.median(np.diff(time_ms))
52     drive = np.empty_like(drives, dtype=np.float32)
53     drive[0] = delay_signal(drives[0], time_ms, W["delay_early"])
54     drive[1:] = delay_signal(drives[1:], time_ms, W["delay_shape"])
55     e, inh = np.zeros_like(drive), np.zeros_like(drive)
56     tau_e = np.array([W["tau_e"], tau_s, tau_s])[: , None]

```

```

57     tau_i = tau_e * np.array([W["tau_i"] / W["tau_e"], W["tau_i_ratio"], W["tau_i_ratio"]])[:, None]
58     for t in range(len(time_ms) - 1):
59         e_in = W["w_ee"] * e[:, :, t] - g_i * W["w_ei"] * inh[:, :, t] + W["g_p"] * drive[:, :, t]
60         i_in = W["w_ie"] * e[:, :, t] - W["w_ii"] * inh[:, :, t] + W["g_q"] * drive[:, :, t]
61         ef, inf = history_at(e[0], t, W["delay_shape"] / dt), history_at(inh[0], t, W["delay_shape"] / dt)
62         e_in[1] += W["feed_e"] * ef
63         e_in[2] += W["feed_e"] * ef.mean()
64         i_in[1] += W["feed_i"] * inf
65         i_in[2] += W["feed_i"] * inf.mean()
66         fe, fi = activation(e_in, 5, 0.35), activation(i_in, 5, 0.35)
67         e[:, :, t + 1] = e[:, :, t] + dt * (-e[:, :, t] + (1 - e[:, :, t]) * fe) / tau_e
68         inh[:, :, t + 1] = inh[:, :, t] + dt * (-inh[:, :, t] + (1 - inh[:, :, t]) * fi) / tau_i
69     return e, inh
70
71
72 def source_currents(e, inh, time_ms):
73     # 兴奋和抑制活动经突触核滤波，再映射为四个半球源和一个中线对手源。
74     dt = np.median(np.diff(time_ms))
75     def kernel(tau):
76         t = np.arange(len(time_ms)) * dt
77         h = t * np.exp(-t / tau) / tau**2
78         return h / (h.sum() * dt)
79     pe = fftconvolve(e, kernel(10)[None, None, :], mode="full", axes=-1)[..., :len(time_ms)] * dt
80     pi = fftconvolve(inh, kernel(20)[None, None, :], mode="full", axes=-1)[..., :len(time_ms)] * dt
81     early, shape, preference = pe - pi
82     return np.stack([early[1], early[0], shape[1], shape[0],
83                     preference[0] - preference[1]])
84
85
86 def leadfield():
87     # 点偶极均匀导体近似，电极位于头表并使用双侧乳突参考。
88     surface = lambda p: RADIUS * p / np.linalg.norm(p, axis=1, keepdims=True)
89     electrodes, references, sources = surface(ELECTRODES), surface(REFERENCES), SOURCES / 1000
90     normals = sources / np.linalg.norm(sources, axis=1, keepdims=True)
91     def potential(points):
92         delta = points[:, None] - sources[None]
93         distance = np.linalg.norm(delta, axis=-1)
94         return np.einsum("sc,esc->es", normals, delta) / (4 * np.pi * SIGMA * distance**3)
95     return potential(electrodes) - potential(references).mean(axis=0, keepdims=True)
96
97
98 def simulate_q2(frontend, time_ms, tau_s=40.0, g_i=1.0, scales=None):
99     e, inh = wilson_cowan(project_features(frontend, scales), time_ms, tau_s, g_i)
100     sources = source_currents(e, inh, time_ms)
101     return leadfield() @ sources

```

C 问题三：认知状态与留记录验证

```

1
2
3 # Q3 动态认知状态模型的名义参数
4 PARAMS = {
5     "tau_visual": 0.060,
6     "tau_memory": 1.0,
7     "tau_control": 0.250,
8     "cue_storage_gain": 0.8,
9     "target_memory_gain": 0.8,
10    "memory_to_control_gain": 0.25,
11    "conflict_gain": 0.8,
12    "topdown_control_gain": 0.10,
13 }
14
15 def make_event_gates(time_s, cue_onset_s=0.0, cue_duration_s=0.203125,
16                     target_onset_s=2.2, target_duration_s=0.2):
17     """根据事件时刻生成提示与目标的阶跃门控信号。"""
18     time_s = np.asarray(time_s, dtype=float).reshape(-1)
19     cue = ((time_s >= cue_onset_s) &
20           (time_s < cue_onset_s + cue_duration_s)).astype(float)
21     target = np.zeros_like(cue)
22     if target_onset_s is not None:
23         if target_duration_s is None:
24             target = (time_s >= target_onset_s).astype(float)
25         else:
26             target = ((time_s >= target_onset_s) &
27                     (time_s < target_onset_s + target_duration_s)).astype(float)
28     return cue, target
29
30 def integrate_macro_states(visual_drive, cue_gate, target_gate, match_evidence,
31                             dt_s, params=PARAMS):
32     """用指数欧拉递推计算视觉、记忆和控制状态。"""
33     drive = np.asarray(visual_drive, dtype=float).reshape(-1)
34     cue = np.asarray(cue_gate, dtype=float).reshape(-1)
35     target = np.asarray(target_gate, dtype=float).reshape(-1)
36     rho_v, rho_h, rho_p = [
37         np.exp(-dt_s / params[key])
38         for key in ("tau_visual", "tau_memory", "tau_control")
39     ]
40     visual = np.zeros_like(drive)
41     memory = np.zeros_like(drive)
42     control = np.zeros_like(drive)
43     visual[0] = drive[0]
44     for k in range(1, drive.size):
45         visual[k] = rho_v * visual[k - 1] + (1 - rho_v) * drive[k]
46         memory_input = (
47             params["cue_storage_gain"] * visual[k] * cue[k]
48             + params["target_memory_gain"] * visual[k] * target[k]
49             * match_evidence
50         )
51         memory[k] = rho_h * memory[k - 1] + (1 - rho_h) * memory_input
52         conflict = target[k] * (1 - match_evidence) / 2
53         control_input = (
54             params["memory_to_control_gain"] * abs(memory[k - 1]) * target[k]
55             + params["conflict_gain"] * conflict
56         )

```

```

57         control[k] = rho_p * control[k - 1] + (1 - rho_p) * control_input
58     return {"visual": visual, "memory": memory, "control": control}
59
60 def observe_macro_model(q2_sensor, memory, control,
61                         memory_loading, control_loading, params=PARAMS):
62     """将 Q2 视觉响应及记忆、控制状态映射到 F3、Fz、F4。"""
63     q2 = np.asarray(q2_sensor, dtype=float)
64     h = np.asarray(memory, dtype=float).reshape(-1)
65     p = np.asarray(control, dtype=float).reshape(-1)
66     l_h = np.asarray(memory_loading, dtype=float).reshape(-1)
67     l_p = np.asarray(control_loading, dtype=float).reshape(-1)
68     return (q2 * (1 + params["topdown_control_gain"] * p)[None, :]
69           + l_h[:, None] * h[None, :]
70           + l_p[:, None] * p[None, :])
71
72 def _features_for_channel(q2_channel, memory, control, interaction):
73     """按通道构造视觉、记忆、控制及视觉反馈特征。"""
74     return np.column_stack((
75         q2_channel.reshape(-1),
76         memory.reshape(-1),
77         control.reshape(-1),
78         interaction.reshape(-1),
79     ))
80
81 def fit_sensor_mapping(observed, q2_sensor, memory, control,
82                       q2_control_interaction, ridge_alpha=1.0):
83     """仅用训练记录拟合各 EEG 通道的岭回归观测映射。"""
84     y = np.asarray(observed, dtype=float)
85     q2 = np.asarray(q2_sensor, dtype=float)
86     h = np.asarray(memory, dtype=float)
87     p = np.asarray(control, dtype=float)
88     interaction = np.asarray(q2_control_interaction, dtype=float)
89     fits = []
90     for channel in range(3):
91         x = _features_for_channel(
92             q2[:, channel], h, p, interaction[:, channel]
93         )
94         target = y[:, channel].reshape(-1)
95         mean_x = x.mean(axis=0)
96         scale_x = x.std(axis=0)
97         scale_x[scale_x < 1e-12] = 1.0
98         z = (x - mean_x) / scale_x
99         mean_y = target.mean()
100         beta = np.linalg.solve(
101             z.T @ z + ridge_alpha * np.eye(z.shape[1]),
102             z.T @ (target - mean_y),
103         )
104         fits.append((mean_x, scale_x, mean_y, beta))
105     return fits
106
107 def predict_sensor_mapping(fits, q2_sensor, memory, control,
108                           q2_control_interaction):
109     """将训练阶段估计的映射应用于留出记录。"""
110     q2 = np.asarray(q2_sensor, dtype=float)
111     h = np.asarray(memory, dtype=float)
112     p = np.asarray(control, dtype=float)
113     interaction = np.asarray(q2_control_interaction, dtype=float)
114     prediction = np.empty_like(q2)
115     for channel, (mean_x, scale_x, mean_y, beta) in enumerate(fits):

```

```

116         x = _features_for_channel(
117             q2[:, channel], h, p, interaction[:, channel]
118         )
119         prediction[:, channel] = (
120             mean_y + ((x - mean_x) / scale_x) @ beta
121         ).reshape(q2.shape[0], q2.shape[2])
122     return prediction
123
124 def score_prediction(observed, predicted, time_s, window):
125     """计算指定时间窗内的 RMSE、标准化 RMSE 与相关系数。"""
126     mask = (time_s >= window[0]) & (time_s < window[1])
127     actual = np.asarray(observed)[..., mask].reshape(-1)
128     estimate = np.asarray(predicted)[..., mask].reshape(-1)
129     rmse = np.sqrt(np.mean((actual - estimate) ** 2))
130     scale = np.std(actual)
131     nrmse = rmse / scale if scale > 1e-12 else np.nan
132     corr = (np.corrcoef(actual, estimate)[0, 1]
133             if np.std(actual) > 1e-12 and np.std(estimate) > 1e-12
134             else np.nan)
135     return {"RMSE": rmse, "NRMSE": nrmse, "相关系数": corr}
136
137 def main():
138     """依次执行事件审计、Q2场景模拟、Q3留记录验证与结果汇总。"""
139     root = str(Path(__file__).resolve().parents[3])
140     if root not in sys.path: sys.path.insert(0, root)
141     event_audit = importlib.import_module("src.C.q3.09_event_time_semantics")
142     validation = importlib.import_module("src.C.q3.dynamic_validation")
143     event_audit.main()
144     return validation.run_validation()
145
146 if __name__ == "__main__":
147     main()

```